

Leveraging Knowledge-Based Techniques to Solve the ARC Challenge

Vignesh Sreedhar
Georgia Institute of Technology
visreedhar@gatech.edu

Abstract

This paper presents a Knowledge-Based AI approach to addressing the Abstraction and Reasoning Corpus (ARC) challenge, leveraging tailored representational structures, rule-based systems, and search and planning strategies. The methodology involves the use of frames and scripts to represent ARC grid states and typical transformation sequences, including but not limited to operations such as mirroring, scaling, and color mapping. The approach then leverages various search and planning methods including Breadth-First Search (BFS), Best First Search, and Local Beam Search to navigate the puzzle state space. The approach scores 7.5 on the eval set, placing third on the student leaderboard.

1. Introduction

The ARC (Abstraction and Reasoning Corpus) Prize is a \$1,000,000+ public competition aimed at promoting artificial general intelligence (AGI) research. It aims to encourage development of AI systems that can efficiently and effectively solve novel problems and hopefully also promote alternative approaches to AGI.

The challenge revolves around the ARC-AGI benchmark, which consists of visual reasoning puzzles that are fairly simple for humans but difficult for current AI systems to solve. Basically, the puzzle provides "training" inputs and outputs with certain patterns and a test input. The goal is for the person/AI system to learn the pattern in the training examples and then ascertain the output given some test input. The challenge in particular tests an AI's ability to acquire new skills and strategies efficiently, rather than relying on pre-trained knowledge, which is the bulk of the large, "intelligent" AI systems today, especially large language models like GPT and Claude.

To solve Abstraction and Reasoning Corpus (ARC) problems efficiently, an AI system would effectively have to reproduce some level of human abstract thinking in order to process and accordingly manipulate information in ways that go beyond simple pattern matching or brute-force

solutions, enabling the creation of generalizable solutions to new problems. The AI must discern patterns from very few examples, so it needs to be aware of structural patterns in the inputs but at the same time be able to identify relationships between elements and apply these relationships in various ways across various problems. Without this abstract reasoning, an AI would struggle with generalization, being able to solve specific cases without understanding the broader principles.

Starting on a more basic level, the AI system needs to be able to process and manipulate the JSON-based input. Each element in the grid is represented by a number (0 for empty and a number for a specific color), and the AI must interpret these numbers and their relationships. The system needs to perform operations like mapping elements to new ones, transforming their positions, or synthesizing new elements altogether based on detected patterns.

Moving to the next level of complexity, the AI system would need to be able to recognize patterns effectively and efficiently; it needs to identify regularities within the grids, such as recurring sequences, symmetries, or relationships between elements. It must identify consistent patterns in how the elements are arranged and determine which features are relevant to solving the problem. For example, detecting that specific numbers always appear in certain rows or are repeated in a symmetrical pattern can help infer transformation patterns.

The AI system would have to be proficient in algorithmic thinking, as these challenges often involve transforming input grids into output grids through a sequence of logical steps. The AI must devise step-by-step procedures to manipulate grid data based on patterns it has detected. In a more technical sense, this could involve translating the numbers in a suitable format, cloning or modeling after sections of the grid, applying geometrical transformations like rotations or reflections, and many other strategies. Importantly, the AI must do this in a structured, logical manner that can be generalized to many examples.

In order to help with efficiency as well, the AI system must be effective at compositional reasoning. The system must be able to break down complex transformations, rules,

and patterns into simpler, more manageable components. For example, an ARC problem might require recoloring elements of a grid and then transforming how its presented via some rotation or translation. The AI system must understand how to break down these transformations into a clear sequence of operations to achieve the desired result.

Building upon the algorithmic thinking, the AI system might need to perform analogical reasoning; when the AI encounters new problems, it should be able to parse through structural similarities with previously solved tasks. Since ARC often presents variations of the same underlying rules across different examples, the AI must be able to map some of the core knowledge from one example to another despite surface-level differences.

Taking it further, inductive reasoning and overall hypothesis generation would also enable the AI to come up with general rules from specific examples and observations. ARC problems typically provide only a few examples to infer the rules governing transformations; as a result, the AI must be able to formulate hypotheses about how the grid transformations work and test these hypotheses on available examples, adjusting them as needed. This could be especially important since we have a limited number of examples for each problem, highlighting the need for generating generalizable hypotheses.

Solving ARC problems requires an AI system with the ability to replicate some level of human reasoning. By combining these cognitive functions, the AI can generalize from a small set of examples, synthesize transformations, and generate executable solutions to a wide range of unseen tasks. This blend of knowledge-based techniques will be the focus of the paper in order to solve the ARC problems efficiently.

2. Related Work

The ARC (Abstraction and Reasoning Corpus) prize poses unique challenges in artificial intelligence by focusing on tasks that demand human-like generalization and abstraction, areas where traditional machine learning approaches often struggle. While there has not been a large body of scholarly work explicitly dedicated to solving ARC tasks, various novel approaches have emerged from both research papers and community discussions, each offering different strategies for addressing the complexity of the ARC.

The ARGa framework discussed in [4] stands out due to its focus on representing images as graphs, enabling a structured, object-centric perspective on ARC tasks. By focusing on the relationships between objects, ARGa mimics how humans often solve abstract reasoning problems by identifying patterns and relationships. The use of graph-based representations allows ARGa to operate at a higher level of abstraction, simplifying the search for correct programs in a

Domain-Specific Language (DSL). Techniques such as constraint acquisition and state hashing further streamline the search process, making it more efficient by pruning irrelevant parts of the solution space and avoiding redundant evaluations. The inclusion of tabu search ensures that ARGa avoids getting stuck in local optima, continually pushing the system to explore new possibilities, making it both robust and flexible in tackling ARC’s inherent complexity.

Similarly, the Generalized Planning for Abstract Reasoning (GPAR) method outlined in [3] takes a different but equally compelling approach by framing ARC tasks as generalized planning problems. By formalizing ARC solutions as compact planning programs with pointers, the GPAR approach benefits from the powerful optimization capabilities of the Planning Domain Definition Language (PDDL). GPAR leverages object-centric abstractions to make ARC tasks manageable within a planning framework, and its reliance on domain-specific knowledge helps scale up the solution process by constraining the search space. This approach allows GPAR to outpace some state-of-the-art solvers on object-centric tasks, proving the flexibility of planning-based models in solving abstract reasoning tasks.

The work on Learning Classifier Systems (LCS) in [1] introduces another intriguing strategy by combining rule-based machine learning with evolutionary algorithms. By evolving a population of condition-action rules, LCS adapts over time, continuously refining its understanding of ARC tasks. The use of genetic algorithms helps ensure diversity in the generated rules, allowing the system to generalize well across multiple tasks. One of LCS’s key strengths is its interpretability—each rule provides a transparent explanation of the system’s decision-making process, which is particularly valuable for tasks like ARC that require human-like reasoning.

Finally, the paper by Ferré [2] introduces a different perspective by applying the Minimum Description Length (MDL) principle to ARC. This method emphasizes simplicity in problem-solving, aiming to find solutions that balance descriptive efficiency with accuracy. By using object-centric models and focusing on reducing the complexity of the rules generated, Ferré’s approach aligns with human cognitive strategies that favor concise, generalizable solutions. The MDL principle not only reduces the risk of overfitting but also promotes models that are interpretable and easy to reason about, which is critical in the context of ARC tasks that demand broad generalization from few examples.

Together, these works represent a wide spectrum of approaches to solving ARC tasks, from graph-based abstraction and planning models to evolutionary algorithms and simplicity-driven models. Each method brings a unique perspective on tackling the challenges of general intelligence and reasoning, demonstrating the potential for interdisciplinary approaches to advance the state of AI in complex

problem-solving domains like ARC.

3. Methodology

To properly address the challenges posed by the ARC Prize, I leverage a combination of tailored representational structures, Knowledge-Based AI methods, and performance and learning mechanisms in order to enable our system to exhibit reasoning and generalization capabilities.

3.1. Representational Structures

- **Frames:** As opposed to semantic networks, I believe that frames would be the best knowledge structure to encapsulate information about specific states and configurations within the ARC problems. Frames are the most analogous to classes and attributes and will thus translate very well in representing a specific ARC grid. The ARC grid frame (class) can include slots (variables) for various attributes and values related to grid elements, allowing the AI to reason about configurations and transformations holistically. This structure enables the system to apply relevant rules more effectively. My approach involves creating a `ARCPuzzle` class to represent each state of the ARC problem.
- **Scripts:** I also leverage scripts, which describe common problem-solving processes encountered in ARC tasks. By outlining typical transformation pathways, my system efficiently navigates through potential solutions by leveraging these established sequences. This approach reduces the cognitive load on the system and streamlines decision-making in complex scenarios.

3.2. Knowledge-Based AI Methods

3.2.1 Rule-Based Systems

A rule-based system serves as the foundation for my problem-solving framework, leveraging production rules to guide transformations and manipulations in the ARC problem space. These transformations include operations like shifting, scaling, cropping, and pattern replication. Each rule below corresponds to a method in the `ARCPuzzle` class and can be systematically applied to generate new configurations from input states. The rules were determined after experimenting with close to fifty examples from the training set that required very different transformations. These rules are by no means exhaustive, and the objective is to include more rules based on the training set.

- `tophalf`: Extracts the upper half of the grid. Useful for symmetrizing operations or isolating specific areas for manipulation.
- `hmirror` and `vmirror`: Reflects the grid horizontally or vertically, effectively flipping rows or columns, often creating symmetry or inversion patterns.

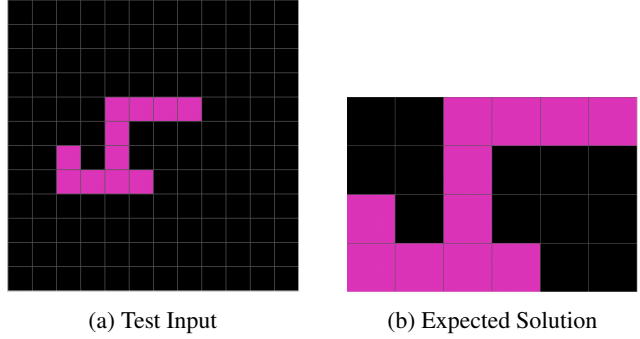


Figure 1: Compress Operation

- `lshift`: Moves non-zero elements in each row to the leftmost positions, padding the remainder of each row with zeros. This rule is beneficial for clustering elements on one side of the grid.
- `compress`: Removes rows or columns where all cells contain the same value, "compressing" the grid to eliminate homogeneous border elements. Figure 1 shows an example of a input and output pair from the training set that requires the `compress` operation to be solved.
- `flood.fill(x, y, target_color)`: Fills a contiguous area starting from the coordinates (x, y) with a specified color, similar to a paint bucket tool. Useful for re-coloring specific regions. Figure 2 shows an example of a input and output pair from the training set that requires the `flood.fill` operation to be solved. Specifically, it requires a `flood.fill` operation on 7 coordinates with the color yellow.
- `crop(x1, y1, x2, y2)`: Crops the grid to include only the sub-region defined by $(x1, y1)$ to $(x2, y2)$, allowing for focused manipulation of specific areas.
- `scale(factor)`: Resizes the grid by the specified scaling factor (e.g., doubling or halving). When scaling up, each element is repeated; when scaling down, elements are skipped accordingly.
- `rotate(degrees)`: Rotates the grid by a specified angle (90° , 180° , or 270°). This is essential for repositioning elements and analyzing rotational symmetry.
- `pattern.repeat(times_horizontal, times_vertical)`: Repeats the grid pattern horizontally and vertically by specified factors, tiling the pattern across a larger grid.
- `alternate_pattern`: Creates an alternating pattern by swapping elements in a checkerboard style,

achieving a distinctive arrangement of rows and columns.

- `mapcolor(a, b)`: Replaces all instances of color *a* in the grid with color *b*, facilitating targeted re-coloring of elements.
- `trim`: Removes a one-cell-wide border from around the grid, typically used to eliminate unwanted boundary elements.

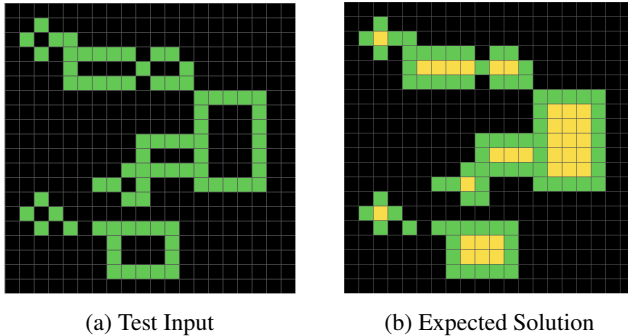


Figure 2: Flood Fill Operation

3.2.2 Search and Planning

In this implementation of the `ARCPuzzle` class, I leverage search and planning techniques to navigate the puzzle state space, transforming the grid from its initial to its goal configuration. To facilitate this, I define an `ARCPuzzleProblem` search problem, and I leverage the `py_search` module introduced in class. I explore a combination of search strategies, including Breadth-First Search, Best First Search, and depth-limited search.

- **Breadth-First Search (BFS)**: I first leveraged Breadth-First Search to systematically explore all possible moves from the initial state in layers, examining all configurations reachable in a given number of moves before moving on to more complex transformations. By following this level-wise exploration, BFS guarantees that the shortest path to the goal state (in terms of number of moves) will be found, as it visits all states within a given depth before expanding further. However, the basic version of BFS was not sufficient since the rules devised in my system were not necessarily sufficient for certain tasks in the evaluation set. Therefore, I apply depth-limiting to restrict the search's reach, which proved particularly useful for tasks that don't require complex, multi-step transformations, allowing for a balance between completeness and computational efficiency. In the end, the combination of Breadth-First Search with depth-limiting

ensured that solutions were generated within defined bounds, ultimately ending for tasks where the rules were not sufficient.

- **Best-First Search (A*) and Heuristic Planning**: Best-First-Search or A* prioritizes states based on a heuristic function that I define. The heuristic function calculates an estimated “cost” or “distance” between the current and goal states. My heuristic calculates the cost from the current node to the goal state by evaluating mismatch scores, and accounting for dimensional differences. I also experimented with accounting mismatches between cells and possible transformations (like resizing, rotating, and mirroring) that may minimize these discrepancies, which helped marginally. This heuristic value is combined with the current path cost, which guides the search towards states that appear closer to the goal. By favoring states with lower heuristic values, I saw improvements in efficiency, often reaching the solution faster than uninformed methods like BFS, but in tasks that were not solvable given the transformations, there was no real improvement. As a result, Best First Search did not prove that much more useful than Breadth-First Search.
- **Local Beam Search**: Local Beam Search was employed as a middle ground between Breadth-First Search and heuristic-driven approaches, focusing exploration on a limited set of the most promising states (defined by the beam). By maintaining a fixed-size beam of the most promising candidate states and expanding only those with the best evaluation scores (based on the above heuristic), the search avoided wasting resources on unpromising paths. To further refine the search and prevent stagnation on plateaus, I introduced a `max_sideways` parameter, which limited the number of consecutive sideways moves—cases where the evaluation score does not improve but exploration continues. This mechanism ensured early termination when no meaningful progress could be made, making the search more efficient for tasks that required exploring beyond the given transformations. While Local Beam Search proved less comprehensive than BFS/A*, its targeted exploration provided a good trade-off between computational efficiency and solution quality in most cases.
- **Planning and Successor Generation**: The `ARCPuzzleProblem` class uses a combination of planning techniques by defining `legalActions()` and `successors()` functions. `legalActions()` lists all feasible actions from a given state, including transformations such as rotations, mirroring, resizing, and color mapping. In order to increase efficiency especially for tasks with grids with big-

ger dimensions, I limited resource-intensive rules like `flood_fill(x, y, target_color)` and `crop(x1, y1, x2, y2)` to states with grids of height and width lower than 20. Each newly generated state is also added to the `visited` set to prevent redundant exploration of already visited states, optimizing the search.

3.2.3 Case-Based Reasoning

I also leverage a very simple Case-Based Reasoning (CBR) approach by choosing the two most common action sequences (if they exist). For each evaluation challenge, the training inputs and outputs represent previous grid states and their desired transformations.

The required actions to convert each input grid to its corresponding output grid are determined using the `arc_planning` function, which basically performs the search and planning mentioned in 3.2.2 and finds a list of actions the goal state if they exist. If two or more different action sequences exist, the two most common action sequences are selected as they represent the most successful solutions.

For each test case, the most frequent action sequences are applied. Attempt 1 uses the most common sequence, while attempt 2 applies the second most common sequence or repeats attempt if only one is available. If no common actions are found, a default grid is used to indicate a lack of similar cases or sufficient rules.

4. Results

Table 1 compares all three search algorithms (with and without practical constraints) that I implemented on the eval tasks: BFS (without depth-limiting), Best-First Search, BFS with depth limiting, and Local Beam Search with the `max_sideways` limit. If any approach took longer than twenty minutes on a single task, I stopped it and calculated the score on the tasks it completed from the evaluation set. The algorithm was run on a Macbook Air with the Apple M1 chip, which has a 3.2 GHz maximum CPU clock rate, 8-core CPU with 4 performance cores and 4 efficiency cores. The table captures each algorithm’s score, total time taken for completion (in minutes), and the number of tasks completed. Here, "DNF" (Did Not Finish) indicates that BFS and Best-First Search could not complete all tasks within the allotted time (spending more than twenty minutes on task 4), finishing only three tasks each. In contrast, the depth-limited BFS completed all 400 tasks in 42.23 minutes or 6.34 seconds per task on average, and the Local Beam Search with the `max_sideways` limit completed all tasks in 3.4 minutes or 1.96 seconds per task on average.

Both BFS and Best-First Search performed similarly in terms of tasks completed and score, each achieving a score

Algorithm	Score	Avg. per (sec)	Time Task	Tasks Completed
BFS	1	DNF		3
Best-First Search	1	DNF		3
BFS with Depth Limit	7.5	6.34		400
Local Beam Search	7.5	1.96		400

Table 1: Performance on Eval Tasks

of zero for the three tasks they completed before reaching the time limit. This outcome suggests that while both algorithms may handle simple task structures effectively, their unrestricted search depths likely led to inefficiency in tackling the larger or more complex task pool, resulting in failure to complete all tasks in a reasonable time. Even though the Best-First Search reached the goal state faster on average, it was not able to stop on time when faced with a task that was probably unsolvable with the given rules.

The depth-limited BFS (depth limit was set to only 2 given slow times), however, achieved a significantly better outcome with a score of 7.5 and completion of all 400 tasks albeit taking more than 6.3 seconds per task. The depth constraint likely allowed it to explore paths more efficiently by limiting its search to a manageable scope, thus avoiding the potential computational overhead encountered by unrestricted BFS and Best-First Search. This finding indicates that setting an appropriate depth limit in search algorithms may enhance efficiency and overall task completion, especially in situations with large task pools or time constraints.

For the Local Beam Search, the `max_sideways` parameter, and the beam width was set to 5 after some fine-tuning. This proves to be the most efficient without compromising on the accuracy. It takes a much lower 1.96 seconds per task on average, and it still scores 7.5 on the eval set. Objectively speaking, however, a score of 7.5/400 suggests that the rules are not sufficient and more work has to be done to build on the existing production rules.

5. Conclusion

My current approach is a great start with a score that places third on the student leaderboard from Gradescope. It provides a solid basis for translating the ARC problem into tangible rules and constraints with a lot of potential for growth. The approach also evaluates the advantages and disadvantages of various search and planning strategies, achieving fairly efficient performance.

There is still a lot of room for improvement. The production rules are clearly not sufficient for a lot of the unseen

tasks, and ultimately, the rules are not generalizable enough for the majority of these tasks. Furthermore, the approach currently leverage a somewhat qualitative approach by evaluating the general transformations required for a proportion of diverse training tasks, but it doesn't actively leverage the training challenges. The approach also does not build upon results from the input and output pairs for each eval task to generate the best final solution apart from simply choosing the most common action sequences.

Given the aforementioned limitations, there are several promising avenues for future exploration:

- **Expanding Production Rules:** The current framework incorporates a variety of transformation rules, such as rotation, scaling, mirroring, flood-fill, and color mapping. Adding more rules, especially those that account for complex pattern recognition or advanced spatial reasoning, could enhance the algorithm's ability to address diverse ARC tasks. Additionally, rules that leverage symmetry, fractal patterns, or other higher-level visual structures might help improve task performance.
- **Experimenting with Alternative Search and Planning Strategies:** The use of BFS, Best-First Search, Local Beam Search provide a solid foundation. However, other optimization and informed search methods, including iterative approaches, may improve efficiency or solution quality by incorporating more dynamic or context-sensitive heuristics. Planning algorithms that factor in learned patterns from the ARC tasks could also offer gains by guiding the search process based on task-specific constraints.
- **Integrating Deep Learning Techniques:** Combining deep learning with rule-based search may enhance the system's adaptability. For instance, a neural network could be trained to learn rules or transformations that are grounded in the production rules and constraints derived from ARC training examples, guiding the search more effectively. This hybrid approach could also leverage deep learning to identify features or patterns that may not be easily captured by deterministic rules alone, thus complementing the strengths of both approaches.

These proposed extensions aim to increase the robustness and flexibility of the approach in tackling the problem, potentially moving the system closer to more generalizable and efficient problem-solving capabilities.

6. Discussion

My approach aligns well with and builds upon several key aspects of the existing related knowledge-based approaches in the ARC field, as highlighted in the related lit-

erature. The distinct aspect of my approach lies in its integration of diverse transformation rules with a search-based strategy, setting a solid foundation for future work, which could incorporate domain-specific learning, further enhancing the model's flexibility and generalizability.

The ARGA framework [4] emphasizes graph-based representations to encapsulate relationships between objects, leveraging a structured, object-centric view to solve ARC tasks. Similarly, my approach includes transformations such as mirroring, scaling, and pattern alternations, which allows to manipulate object relationships within the grid. While my approach does not explicitly use a graph-based model, the variety of operations and state transformations reflect an implicit understanding of object relationships, suggesting a conceptual parallel with ARGA. Both approaches prioritize abstraction and the manipulation of object-centric properties, allowing complex configurations to be represented and managed in a structured way. GPAR [3] frames ARC solutions within a planning framework, benefiting from the efficiency of compact planning programs. My method, with its focus on transformations and search-based planning, similarly leverages structured sequences of transformations to reach solutions. This suggests that, like GPAR, my approach could benefit from integrating more domain-specific constraints or optimizations to reduce the search space. Additionally, exploring generalized planning techniques, potentially informed by learned constraints as proposed in my future work, may make my approach even more efficient and specialized. My approach also draws parallels with the LCS method [1], which uses rule-based learning enhanced by evolutionary algorithms to generate and refine decision rules over time. By implementing a diverse set of deterministic transformations, my system forms a rule-based framework similar to LCS. Combining deep learning to guide rule generation could therefore further strengthen this connection by dynamically adjusting or evolving rules, enabling a system that continuously adapts to ARC's requirements.

On a more general note, the results of my approach contribute to the field of Knowledge-Based AI by highlighting the practicality and limitations of rule-based systems when applied to abstract and complex reasoning tasks like ARC. Specifically, the integration of tailored representational structures and deterministic transformations demonstrates the value of grounding problem-solving strategies in well-defined production rules. However, my findings also challenge the notion that purely rule-based methods can generalize effectively across diverse problem spaces. While traditional methods like BFS, A*, and even beam search guide the search fairly effectively, it seems like a hybrid approach combining deterministic rules with adaptive learning may be necessary. This work introduces the potential for combining symbolic AI methods, such as frames and

scripts, with data-driven learning approaches to bridge the gap between deterministic reasoning and pattern recognition, paving the way for more robust, adaptable AI systems. Furthermore, by leveraging search constraints and case-based reasoning, this methodology offers practical insights into optimizing rule-based systems for large-scale problem domains, presenting a step forward in developing flexible Knowledge-Based AI frameworks.

References

- [1] Cameron Coombe, David Howard, and Will Browne. Learning classifier systems as a solver for the abstraction and reasoning corpus. 2024. [2](#), [6](#)
- [2] Sébastien Ferré. Tackling the abstraction and reasoning corpus (arc) with object-centric models and the mdl principle. In *Advances in Intelligent Data Analysis XXII, IDA 2024*, pages 3–15. Springer, 2024. [2](#)
- [3] Chao Lei, Nir Lipovetzky, and Krista A. Ehinger. Generalized planning for the abstraction and reasoning corpus. 2024. [2](#), [6](#)
- [4] Yudong Xu, Elias B. Khalil, and Scott Sanner. Graphs, constraints, and search for the abstraction and reasoning corpus. 2022. [2](#), [6](#)